

Clase 261 — Seguridad de Android: arquitectura

Parte: **13 — Seguridad móvil, IoT e inalámbrica** · Fuente: *The Mobile Application Hacker's Handbook* (Chell et al.) y OWASP MASTG 🕒 Duración estimada: **90 min** · Nivel: **Intermedio**

Objetivo

Comprender el modelo de seguridad de Android de extremo a extremo —desde el kernel Linux y SELinux hasta el sandbox de aplicaciones, el modelo de permisos y el almacenamiento de claves— para poder razonar sobre qué protege el sistema, dónde están sus límites y qué asunciones rompe un dispositivo rooteado. Esta clase es la base conceptual sin la cual el pentest de la siguiente clase sería mera repetición de comandos.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Explicar** las capas de la pila Android (kernel, HAL, ART, framework, apps) y su rol de seguridad.
2. **Describir** cómo el sandbox por UID y SELinux aíslan aplicaciones entre sí.
3. **Analizar** el modelo de permisos (install-time, runtime, permisos peligrosos) y sus abusos.
4. **Identificar** dónde y cómo Android almacena secretos (Keystore, `SharedPreferences`, cifrado de disco).
5. **Enumerar** los componentes de una app (Activities, Services, Broadcast Receivers, Content Providers) y su exposición.
6. **Evaluar** el impacto de rotear un dispositivo sobre las garantías del modelo.

Temas

#	Tema	Por qué importa
1	Pila Android y arranque verificado	Define la cadena de confianza desde el bootloader
2	Sandbox por UID y SELinux	Es el aislamiento primario entre apps
3	Modelo de permisos	Controla acceso a datos y hardware sensibles
4	Componentes de aplicación e IPC (Intents)	Superficie de ataque expuesta por el desarrollador
5	Android Keystore y almacenamiento	Dónde viven las claves y por qué a veces se filtran
6	Firma de APK y actualización	Garantiza integridad y origen del código
7	Root y su impacto en el modelo	Rompe casi todas las asunciones defensivas

Definiciones y características

- **Sandbox de aplicación:** cada app corre con un UID Linux único; el kernel impide que una acceda a los ficheros de otra. Característica clave: aislamiento por proceso, no por confianza en el código.

- **SELinux (enforcing):** control de acceso obligatorio (MAC) que confina incluso a `root` mediante políticas. Característica: reduce el impacto de una escalada.
- **Permiso peligroso (dangerous):** permiso que da acceso a datos privados (contactos, ubicación, cámara) y requiere consentimiento del usuario en runtime desde Android 6. Característica: revocable por el usuario.
- **Android Keystore:** almacén de claves respaldado por hardware (TEE/StrongBox) donde la clave nunca sale del entorno seguro. Característica: operaciones criptográficas sin exponer el material.
- **Content Provider:** componente que expone datos estructurados vía URI `content://`. Característica: si está `exported` sin permisos, otras apps leen sus datos.
- **APK signing (v2/v3):** firma del paquete que garantiza que no fue alterado desde su publicación. Característica: la clave del desarrollador ancla la identidad de la app.

Herramientas y preparación

- **Android Studio + AVD** (emulador) o un dispositivo físico de pruebas **de tu propiedad**.
- `adb` (Android Debug Bridge), incluido en platform-tools.
- Emulador con imagen *Google APIs* (no *Play Store*) para obtener acceso root fácilmente.
- Opcional: dispositivo con **Magisk** para root en hardware real, solo en equipo propio.

```
# Verificar conexión y explorar el sistema
adb devices
adb shell getprop ro.build.version.sdk      # nivel de API
adb shell getenforce                        # estado de SELinux (Enforcing/Permissive)
adb shell id                               # UID/contexto del shell
```

 Trabaja únicamente sobre emuladores o dispositivos propios dedicados a laboratorio.

Laboratorio guiado

1. **Levanta un AVD** con API 30+ e imagen *Google APIs*. Arráncalo desde Android Studio o con `emulator -avd <nombre> -writable-system`.
2. **Inspecciona el sandbox:** instala una app de prueba y localiza su UID: `adb shell pm list packages -U | grep <paquete>`. Observa que su directorio `/data/data/<paquete>` no es legible por otras apps.
3. **Revisa SELinux:** ejecuta `adb shell getenforce`. Consulta el contexto de un proceso con `adb shell ps -Z`.
4. **Enumera permisos** de una app instalada: `adb shell dumpsys package <paquete> | sed -n '/requested permissions/,/install permissions/p'`.
5. **Examina componentes exportados:** extrae el APK (`adb shell pm path <paquete> + adb pull`) y ábrelo con `aapt dump xmltree base.apk AndroidManifest.xml` buscando `android:exported="true"`.
6. **Explora el Keystore:** con una app de ejemplo que genere una clave, observa que el material no aparece en el sistema de ficheros de la app.
7. **Rootea el emulador:** `adb root` (en imágenes de ingeniería) y comprueba cómo ahora `adb shell` accede a `/data/data/*` de cualquier app: constata que el root anula el sandbox.

Ejercicios

1. Dibuja un diagrama de la pila Android indicando en qué capa vive cada control de seguridad.
2. Lista cinco permisos "peligrosos" y explica qué dato o hardware protege cada uno.
3. Compara `SharedPreferences` en claro vs. `EncryptedSharedPreferences` y di cuándo usar cada uno.
4. Identifica en un APK real (propio) todos los componentes `exported` y clasifica su riesgo.
5. Explica con tus palabras por qué SELinux mitiga una vulnerabilidad de escalada a root.
6. Investiga la diferencia entre TEE y StrongBox para el respaldo del Keystore.

Reto verificable

Toma una app de prueba (por ejemplo, una que tú compiles) y produce un **inventario de superficie de ataque local**: tabla con todos sus componentes, su estado `exported`, permisos requeridos y dónde guarda datos. **Criterio de aceptación**: el inventario identifica al menos un Content Provider o Activity exportada y justifica si representa o no un riesgo, citando la evidencia extraída del manifiesto.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>adb: no devices/emulators found</code>	El servidor adb no ve el AVD; reinícialo con <code>adb kill-server</code> && <code>adb start-server</code>
<code>adbd cannot run as root in production builds</code>	Imagen <i>Play Store</i> ; usa una imagen <i>Google APIs</i> o de ingeniería
<code>Permission denied</code> al leer <code>/data/data</code>	El sandbox lo impide sin root; usa un AVD rooteable
<code>getenforce</code> devuelve <code>Permissive</code>	Imagen de desarrollo; en producción SELinux está en <code>Enforcing</code>
Manifiesto ilegible tras <code>apktool</code>	Falta reconstruir recursos; usa <code>aapt dump xmltree</code> sobre el APK original

Preguntas frecuentes

¿El sandbox de Android me protege aunque instale una app maliciosa? Limita su acceso a otras apps y al sistema, pero la app maliciosa sigue pudiendo abusar de los permisos que le concedas y de componentes mal expuestos por otras apps.

¿Rootear siempre debilita la seguridad? Sí: rompe el sandbox, permite a apps con root leer datos de cualquier otra y desactiva garantías del arranque verificado. En laboratorio es útil; en un teléfono personal es un riesgo real.

¿Dónde deberían guardarse las claves criptográficas de una app? En el Android Keystore respaldado por hardware, nunca en `SharedPreferences` ni hardcodeadas en el código.

Referencias

- OWASP MASTG — Android Platform Overview: <https://mas.owasp.org/MASTG/0x05a-Platform-Overview/>

- Android Security Model (documentación oficial): <https://source.android.com/docs/security>
- Android Keystore System: <https://developer.android.com/training/articles/keystore>
- *The Mobile Application Hacker's Handbook*, cap. Android — Chell, Erasmus, Colley, Whitehouse.

Siguiente clase

Clase 262 - Pentest de aplicaciones Android